

Real-Time Virtual Mentor for Introductory Programming Feedback

Alimzhan Koshikov

Master Degree Student, School of Software Engineering
Astana IT University (AITU)
Astana, Kazakhstan
225148@astanait.edu.kz
0009-0003-3234-6487

Tutkyshbayeva Shyrin

PhD, School of Software Engineering
Astana IT University (AITU)
Astana, Kazakhstan
sh.tutkyshbayeva@astanait.edu.kz
0000-0002-1419-2571

Abstract—Introductory programming is often a challenging experience for CS students, especially for first-year students. In various CS1 (introductory programming courses) courses, feedback is often delayed, limited, or too generic, and because of this students repeatedly face syntax and logic errors without clearly understanding how to solve them. In order to solve this problem, this paper proposes a virtual mentor for CS1 that provides real-time tailored feedback during the problem-solving process. The proposed system combines continuous code analysis, a lightweight student model, and a pedagogical policy layer. Such a combination makes it possible to detect common error patterns, track repeated mistakes, take previous feedback history into account, and decide when and how much guidance should be given in order to support student independence and critical thinking. Compared to other assistants with no constraints, the virtual mentor can decide what type of feedback is most helpful for student in a particular situation, such as guiding questions, conceptual hints, code highlighting, or no feedback, while keeping response latency low. In a pilot prototype evaluation session based on 78 event-level interaction logs, the system demonstrated interactive responsiveness during the session, with a mean end-to-end latency of 740 ms and 800 ms at P95. During the same pilot trace, it showed an event-level success rate of 70% and a median time-to-fix of 2.7 s. These results suggest that real-time personalization with explicit pedagogical constraints can provide practical and scalable support in CS1 settings.

Index Terms—Virtual mentor, CS1, introductory programming, programming education, real-time feedback, student modeling, pedagogical constraints

I. INTRODUCTION

Introductory programming (CS1) remains difficult for first-year students because they must simultaneously learn syntax, develop semantic understanding, and build problem-solving habits. In large classes, beginners often repeat the same compiler and logic errors, while instructors and teaching assistants cannot always provide immediate and individualized support at scale [1], [2].

Automated assessment improves scalability through compiler messages, unit tests, and static checks, but such feedback is often binary or generic and does not explicitly address misconceptions or the level of support a student actually needs. Prior work in computing education shows that feedback quality depends not only on correctness, but also on timing, pedagogical framing, and whether the intervention preserves

student autonomy rather than simply producing a working answer [1], [3].

Recent AI-based teaching assistants have shown that near real-time support is feasible in large-scale educational settings. For example, the CS50 AI assistant reported high adoption and low operating cost at MOOC scale [4]. At the same time, prior studies also highlight important risks: students may overuse assistants for immediate help, explanations may be inaccurate, and higher perceived usefulness does not necessarily translate into better learning outcomes [2], [3], [5]. These findings suggest that educational assistants should not only generate responses, but should also regulate *when* help is given, *what kind* of help is given, and *how much* information is disclosed [1], [6], [7].

This paper presents a virtual mentor for CS1 that is designed as a bounded real-time decision pipeline rather than as a general conversational assistant. The system combines continuous code analysis, a lightweight student state, and a pedagogical policy layer that selects an intervention type under latency constraints. If a language model is used, it is treated only as a realization component for phrasing feedback, not as the core decision-making mechanism.

The main contribution of this work is not another generic AI coding assistant, but a prototype architecture for *policy-guided, real-time mentoring* during active programming. In contrast to systems that respond only after submission or produce unconstrained natural-language help, the proposed mentor explicitly separates analyzer signals, student state tracking, action selection, and feedback realization.

The main contributions of this work are as follows:

- a modular virtual mentor architecture that consists of modules such as real-time code analysis, lightweight student-state tracking, pedagogical action selection, and feedback realization;
- a policy-guided feedback mechanism that regulates timing and detail of help while avoiding direct solution disclosure;
- a pilot prototype evaluation based on event-level interaction logs, demonstrating bounded-latency operation and preliminary action-level effectiveness trends.

TABLE I
POSITIONING OF THE PROPOSED VIRTUAL MENTOR AGAINST REPRESENTATIVE PROGRAMMING FEEDBACK SYSTEMS

System	Core Technology	Main Feedback Mode	Reported Focus / Metric
AIF 3.0 [8]	Hybrid data-driven + expert-based	Progress indicators and subgoal-oriented feedback	Subgoal detection accuracy: 88%
CodeAid [6]	GPT-3.5-Turbo	Pseudocode-oriented explanations and guided support	Technical correctness: 79%
Verifix [9]	Program analysis	Verified repair suggestions	Feedback quality score: 3.8/5.0
DeepFix [9]	Neural error-correction model	Syntax error correction	Full error correction rate: 27%
Proposed Virtual Mentor	Real-time analyzer + student state + policy layer + optional LLM realization	Pedagogically constrained in-process feedback during code writing	Pilot prototype results reported in Sec. V

II. RELATED WORK

Research on AI support for programming education commonly falls into three broad directions: intelligent tutoring systems, automated feedback generation, and LLM-based assistants. The proposed virtual mentor draws from all three, but combines them with a different emphasis on bounded-latency interaction and policy-level control.

A. Intelligent Tutoring Systems for Programming

Intelligent tutoring systems (ITS) personalize instruction by modeling learner knowledge and adapting feedback. Modern systems increasingly focus on augmentation and adaptive sequencing rather than full automation. ProgTutor, for example, combines adaptive sequencing with automated evaluation while keeping instructors in the loop [10]. A recurring limitation is interaction granularity: many ITS intervene at discrete steps or after submission, which restricts timely support during active code construction.

Our virtual mentor targets the in-process setting with continuous monitoring and timely pedagogical intervention.

B. Automated Feedback Generation Systems

Automated feedback is frequently used in CS1 courses for scalability and usually by the use of unit tests, static analysis and compiler diagnostics. Although such type of feedback is simple, fast and effective for quick checks, but it often superficial and binary, and it does not consider student’s misconceptions. Kumar et al. said that majority of such tools checks only for syntax or correctness of output rather than providing conceptual and tailored guidance, while reproducibility and adaptation remain open issues [9].

These limitations help understand that solution is to combine code analysis with student modeling and adaptive pedagogical policies, as done in our architecture.

C. LLM-Based Programming Assistants

Compared to rule-based feedback systems, LLMs allow natural-language explanations and hints, provide higher flexibility [3]. Assistants with pedagogical constraints (pedagogical control layer) such as CodeAid and Iris confirm that LLM-based assistance can be effective when it designed to stimulate student’s independence and critical thinking, rather than providing solutions directly [6], [7]. At the same time, classroom evidence

highlights important risks. Students may seek immediate answers instead of engaging with the underlying concept, and the assistant may still produce explanations that are inaccurate, overly strong, or poorly aligned with course objectives [2], [5]. At larger scale, CS50 also emphasizes the need for continuous human feedback and systematic evaluation to maintain pedagogical alignment [4].

D. Research Gap and Positioning

Existing work does not consistently combine three requirements in a single CS1 support system: (i) intervention during active code writing rather than only after submission, (ii) an explicit lightweight student state that captures repeated difficulties, and (iii) a pedagogical policy that regulates feedback timing, type, and disclosure level. Intelligent tutoring systems provide personalization, but they often rely on heavier domain modeling or operate at coarser interaction granularity. LLM-based assistants offer fluent explanations, but many of them function as open-ended help tools without explicit student-state tracking or controlled intervention logic [6], [7], [9], [10].

The virtual mentor proposed in this work is positioned between these two lines of research. It does not attempt to replace instructors, nor does it rely on unconstrained response generation. Instead, it treats mentoring as a bounded-latency decision process: analyzer signals are combined with a lightweight student state, a policy selects an action, and the selected action is then realized as feedback. This design shifts the technical focus from generic answer generation to controlled pedagogical intervention.

Table I summarizes representative systems and highlights the positioning of the proposed virtual mentor relative to prior approaches.

III. VIRTUAL MENTOR SYSTEM ARCHITECTURE

The proposed virtual mentor is a real-time decision pipeline that maps a stream of student code states to pedagogically constrained feedback actions. The architecture separates code analysis, student-state tracking, policy selection, and feedback realization in order to improve modularity, simplicity and keep latency low.

Contrary to standalone LLM assistants that act as an external helper, the mentor is integrated into the programming session and gradually observes code changes. Natural language

generation is used only at the realization stage as a final product of the analysis process, while core decision logic works beyond the natural language model and is driven by combination of analyzer, policy engine and student state manager.

A. Data Flow

As shown in Fig. 1, code edits are processed by an analyzer that performs syntactic checks, lightweight structural analysis, and detection of common novice error patterns. The analyzer output is passed to a profile manager that maintains a compact student state, including repeated error categories and recent feedback history. A policy layer then selects the appropriate feedback action, and the selected action is realized as a concrete intervention, such as a guiding question, a conceptual hint, or a code-region highlight.

This separation is important for two reasons. First, it allows the system to remain responsive under real-time constraints by keeping the expensive components outside the critical decision path. Second, it makes the intervention logic explicit, which is useful both for pedagogical control and for future comparison between rule-based and learned policy variants.

B. Formal System Model

Let c_t denote the student code state after an edit at time t . The analyzer

$$A : c_t \rightarrow e_t$$

maps c_t to an error descriptor $e_t \in \mathcal{E}$, where $\mathcal{E}$ is a finite set of novice-relevant error categories.

The student state is represented as a lightweight vector

$$s_t = (n_{e_1}, n_{e_2}, \dots, n_{e_k}, h_t),$$

where n_{e_i} is the cumulative count of error type e_i observed up to time t , and h_t denotes compact history features such as the most recent intervention type.

Given (e_t, s_t) , the policy selects an action

$$\pi : (e_t, s_t) \rightarrow a_t,$$

where $a_t \in \mathcal{A}$ and

$$\mathcal{A} = \{\text{GQ}, \text{CH}, \text{HL}, \text{NF}\}$$

corresponds to *Guiding Question*, *Conceptual Hint*, *Code Highlight*, and *No Feedback*.

In the current prototype, π is available in two variants: a rule-based baseline and a learned policy extension. In the learned variant, action selection is formulated as supervised multi-class classification:

$$\hat{a} = \arg \max_{a \in \mathcal{A}} P(a | x),$$

with feature vector x constructed from the current error category, recurrence counts, and recent interaction context.

In the current prototype, this interface includes compile status, novice error category, recurrence count, recent feedback history, and suspicious-region metadata.

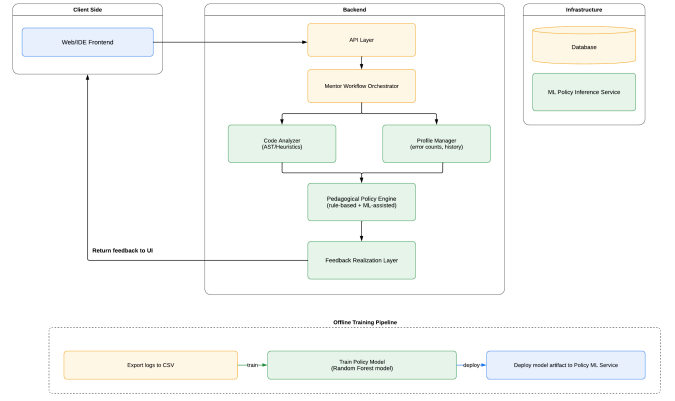


Fig. 1. High-level architecture of the virtual mentor system.

C. ML-Based Policy Extension

In addition to the rule-based baseline, the current prototype includes a learned policy extension for pedagogical action selection. This component predicts one of four feedback actions — `CODE_HIGHLIGHT`, `CONCEPTUAL_HINT`, `GUIDING_QUESTION`, or `NO_FEEDBACK` — from analyzer-derived and session-derived features.

To bootstrap this learned component, we used an external programming corpus derived from the public Kaggle dataset - *Coding Questions with Solutions*. The dataset provides programming questions together with solution-oriented material and associated test cases, and spans multiple difficulty levels, including introductory and more advanced tasks. Because this corpus does not consist of authentic classroom interaction traces, it was used as an auxiliary source for constructing training examples for the policy model rather than as a substitute for real student mentoring data.

The learned policy was implemented as a multi-class Random Forest classifier using `scikit-learn`. The model was configured with 200 trees, `max_depth=8`, `min_samples_leaf=2`, `class_weight="balanced_subsample"`, and `random_state=42`. The input feature space combines analyzer-level and session-level attributes, including error type, severity, compile success, test outcomes, recurrence statistics, attempt number, previous feedback action, previous feedback success, suspicious-region presence, code length, and cumulative feedback count within the session.

Before training, the feature space was processed through a tabular preprocessing pipeline. Categorical variables were imputed with a constant placeholder and one-hot encoded, boolean variables were imputed using the most frequent value, and numeric variables were imputed using median values. This design allows the policy model to operate over heterogeneous mentoring signals while preserving the same interface used by the rule-based policy.

D. Core Components

Analyzer: Processes incoming code states and extracts pedagogically relevant signals, including compile status, novice error categories, and suspicious code regions.

Profile Manager: Maintains a lightweight and interpretable student state with low computational overhead. The design favors responsiveness and transparency over heavy knowledge-tracing machinery.

Policy Layer: Chooses whether and how the system should intervene. This component is distinct from code analysis: it does not detect errors itself, but decides how analyzer signals should be translated into pedagogical action.

Feedback Realization: Produces the final response corresponding to the chosen action. This may be implemented through templates or a language model, but in both cases the output is constrained by the selected action type and the no-solution-disclosure rule.

IV. REAL-TIME FEEDBACK MECHANISM

The feedback mechanism is designed for supporting students during active coding session rather than post-submission support. Previous works on LLM-based programming assistants demonstrates that real-time, immediate help is helpful, but untimely assistance may lead to trial-and-error behavior and reduce critical thinking and depth of engagement in problem solving [5]. In order to solve this problems, the proposed system does not react for every detected issue as a trigger for the same type of response.

A. Feedback Generation

When a learner modifies code, the analyzer first produces a short description of the current state, including error-related signals, compile status, and possible suspicious regions. These signals are recorded in the student state, allowing the system to distinguish between new difficulties and repeated ones. The policy layer then selects one of four intervention types: a guiding question, a conceptual hint, a code highlight, or no feedback. The selected action is finally realized as student-facing feedback in the interface.

This design reflects the pedagogical assumption that not all support should be equally explicit. In some cases, highlighting a suspicious region is sufficient. In others, repeated difficulties may justify a conceptual hint. A guiding question is used when the system aims to prompt reasoning without directly revealing the next implementation step. Complete solutions are intentionally avoided.

Fig. 2 shows the main elements of the interface involved in the problem solving session, including the task card, the code editor, the mentor feedback panel, the analyzer metadata, and the feedback-loop controls.

B. Timing Constraints

To maintain interactivity, the total response time is constrained by

$$T_{total} = T_{analysis} + T_{policy} + T_{feedback},$$

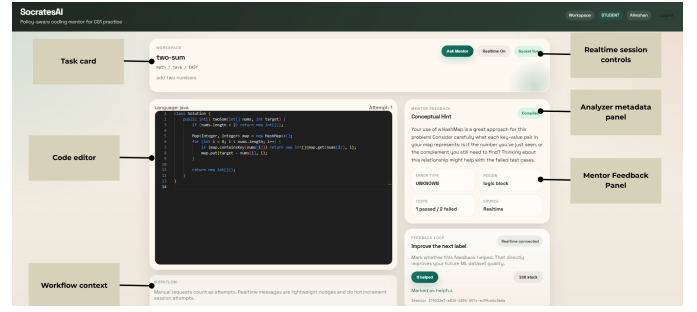


Fig. 2. Interface with the main functional components of the real-time mentoring workflow

where $T_{analysis}$ denotes analyzer processing time, T_{policy} denotes action-selection time, and $T_{feedback}$ denotes feedback realization time.

The prototype is designed so that the policy stage remains lightweight and does not dominate the latency budget. In the pilot interaction logs, the end-to-end latency remained within interactive bounds, with mean $T_{total} = 740$ ms and P95 800 ms. The mean component times were 618 ms for analysis, 26 ms for policy selection, and 12 ms for feedback realization, with the remaining delay attributable to platform overhead.

V. EXPERIMENTAL EVALUATION

A. Evaluation Artifact

The current evaluation is based on a pilot event-level interaction log produced by the prototype rather than on a completed classroom-scale comparative study. The analyzed artifact contains 78 logged interaction events collected during representative CS1 problem-solving sessions. Each event records analyzer output, selected feedback action, latency measurements, and whether the detected issue was resolved after the intervention.

The learned policy component should be distinguished from the pilot evaluation artifact. Model training relied on an external programming corpus used to construct policy-level training examples, whereas the present paper evaluates the prototype through event-level interaction logs generated by the system itself. This separation is intentional: the external corpus supports policy development, while the reported pilot results reflect operational behavior of the mentoring pipeline.

At the current stage, the available artifact is more suitable for system-level and event-level analysis than for participant-level educational inference. For this reason, the paper reports responsiveness and action-level effectiveness trends, but does not claim statistically validated learning gains, longitudinal retention effects, or benchmark superiority over external systems. If student- and task-level distributions are later consolidated into a broader classroom dataset, they should be reported in a larger follow-up evaluation.

Each logged interaction is represented as

$$x_i = (e_i, c_i, t_i^{analysis}, t_i^{policy}, t_i^{feedback}, a_i, s_i),$$

TABLE II
PILOT RESULTS FROM EVENT-LEVEL INTERACTION LOGS

Metric	Value
Events (N)	78
Mean latency $\bar{T}$ (ms)	740
P95 latency (ms)	800
Mean analysis/policy/feedback (ms)	618 / 26 / 12
Platform overhead (ms)	85
Event success rate (%)	70.0
Median time-to-fix (ms)	2700
P95 time-to-fix (ms)	3600

TABLE III
EVENT-LEVEL RESOLUTION BY FEEDBACK ACTION

Action	Events (N)	Success rate (%)
CODE_HIGHLIGHT	23	82.6
CONCEPTUAL_HINT	26	80.8
GUIDING_QUESTION	21	57.0
NO_FEEDBACK	8	25.0

where e_i denotes the current error category, c_i the recurrence count, $t_i^{analysis}$, t_i^{policy} , and $t_i^{feedback}$ the measured latency components, a_i the selected feedback action, and s_i the event outcome indicator.

B. Metrics

Two groups of metrics were considered. First, responsiveness was measured through end-to-end latency and its component breakdown:

$$\bar{T} = \frac{1}{N} \sum_{i=1}^N T_{total}^{(i)}.$$

Second, we used event-level effectiveness proxies derived from the interaction logs, including event success rate and time-to-fix. In this context, an event is counted as successful if the issue associated with the intervention was resolved after the selected feedback action. These metrics provide an operational view of the prototype, but they should not be interpreted as full learning-outcome measures.

C. Results

Table II summarizes the pilot results. The main purpose of these results is to show that the proposed architecture can operate within interactive latency constraints while producing different intervention types in a functioning mentoring loop.

VI. DISCUSSION

A. Interpretation of Results

The pilot results suggest that the proposed mentoring loop can operate under interactive time constraints. With a mean end-to-end latency of **740** ms and P95 of **800** ms, the prototype remained within a response range that is compatible with in-process feedback during code writing. This is important because the main technical goal of the system is not only to generate help, but to do so without disrupting the programming flow.

The action-level breakdown also provides an initial indication that different intervention types may have different short-term

effects. In the current trace, code highlighting and conceptual hints were associated with higher immediate resolution rates than guiding questions. This does not yet justify strong causal claims, but it does support the idea that feedback type matters and that action selection should be treated as an explicit design problem rather than as a by-product of response generation.

More broadly, the pilot supports the feasibility of the architectural separation used in this work. The analyzer, student state, policy layer, and feedback realization operated as distinct components inside one real-time loop, which is the main systems contribution of the prototype.

B. Limitations

This study has several limitations. First, the present evaluation is based on a pilot interaction log with a limited number of events and should therefore be interpreted as prototype-level evidence rather than classroom-scale validation. Second, the current artifact does not provide inferential comparison against external benchmark systems or against a completed control-group study. Third, the reported effectiveness measures are short-term event-level proxies and do not establish learning gains, retention, or transfer. An additional limitation is that the learned policy was bootstrapped from an external programming corpus rather than from a large collection of authentic student-mentor interaction traces. Although this was sufficient for prototyping the ML-based action selector, broader classroom data will be needed to determine whether the learned policy generalizes reliably to real CS1 mentoring settings. Finally, the student state remains intentionally lightweight; richer learner modeling may improve adaptivity, but would also increase implementation complexity and data requirements.

C. Practical Implications

Despite these limitations, the results indicate that real-time personalized support can be introduced as a middle layer between static automated checks and human instruction. In this role, the mentor does not replace instructors or teaching assistants. Instead, it helps filter routine support situations, provides bounded assistance during problem solving, and preserves instructional intent through explicit control of intervention type.

D. Ethical Considerations

The system is designed to support student autonomy by avoiding direct solution disclosure and prioritizing formative guidance. Interaction logs are stored for system analysis and should be anonymized in real deployments. Since AI-assisted help may encourage over-reliance if used without safeguards, the policy layer is intended to regulate feedback timing and explicitness rather than maximize immediate answer delivery.

VII. FUTURE WORK

Several directions remain for future work. First, the prototype should be evaluated on a larger and more diverse set of CS1 tasks, including comparative baselines such as non-personalized automated feedback and fixed rule-only intervention policies. Second, the current learned policy model can be validated

and refined on larger collections of authentic student–mentor interaction traces while preserving the same feature interface. Third, broader classroom studies are needed to test whether short-term event-level effectiveness translates into measurable learning outcomes, retention, and transfer. Finally, the student-state representation can be extended beyond simple recurrence counts to include temporal learning dynamics and teacher-in-the-loop adjustment of policy behavior.

VIII. CONCLUSION

This paper presented a bounded-latency virtual mentor for introductory programming that combines real-time code analysis, lightweight student-state tracking, pedagogical action selection, and constrained feedback realization.

The pilot prototype evaluation showed that the system can operate within interactive latency limits, with mean end-to-end latency of **740 ms** and P95 of **800 ms**. Across **78** event-level interactions, the prototype achieved an event success rate of **70%** and a median time-to-fix of **2.7 s**. These results should be interpreted as feasibility evidence rather than as final learning-outcome validation, but they support the practical viability of policy-guided, real-time mentoring for CS1.

The main contribution of the work lies in the architectural separation of analyzer signals, student-state tracking, policy-based intervention, and feedback realization under bounded latency. This provides a concrete foundation for future comparative and classroom-scale studies.

ACKNOWLEDGMENT

This work was carried out as part of the master’s degree program at Astana IT University. The authors thank Astana IT University for providing the academic environment and computing infrastructure that supported the development of the prototype.

REFERENCES

- [1] P. Denny, S. MacNeil, J. Savelka, L. Porter, and A. Luxton-Reilly, “Desirable characteristics for ai teaching assistants in programming education,” pp. 408–414, 2024. [Online]. Available: <https://doi.org/10.1145/3649217.3653574>
- [2] U. Z. Ahmed, S. Sahai, B. Leong, and A. Karkare, “Feasibility study of augmenting teaching assistants with ai for cs1 programming feedback,” pp. 11–17, 2025. [Online]. Available: <https://doi.org/10.1145/3641554.3701972>
- [3] A. F. Pereira and R. Ferreira Mello, “A systematic literature review on large language models applications in computer programming teaching evaluation process,” *IEEE Access*, vol. 13, pp. 113 449–113 460, 2025. [Online]. Available: <https://doi.org/10.1109/ACCESS.2025.3584060>
- [4] R. Liu, J. Zhao, B. Xu, C. Perez, Y. Zhukovets, and D. J. Malan, “Improving ai in cs50: Leveraging human feedback for better learning,” pp. 715–721, 2025. [Online]. Available: <https://doi.org/10.1145/3641554.3701945>
- [5] B. Sheese, M. Liffiton, J. Savelka, and P. Denny, “Patterns of student help-seeking when using a large language model-powered programming assistant,” pp. 49–57, 2024. [Online]. Available: <https://doi.org/10.1145/3636243.3636249>
- [6] M. Kazemitabaar, R. Ye, X. Wang, A. Z. Henley, P. Denny, M. Craig, and T. Grossman, “Codeaid: Evaluating a classroom deployment of an llm-based programming assistant that balances student and educator needs,” 2024. [Online]. Available: <https://doi.org/10.1145/3613904.3642773>
- [7] P. Bassner, E. Frankford, and S. Krusche, “Iris: An ai-driven virtual tutor for computer science education,” pp. 394–400, 2024. [Online]. Available: <https://doi.org/10.1145/3649217.3653543>
- [8] S. Marwan, B. Akram, T. Barnes, and T. W. Price, “Adaptive immediate feedback for block-based programming: Design and evaluation,” *IEEE Transactions on Learning Technologies*, vol. 15, no. 3, pp. 406–420, 2022. [Online]. Available: <https://doi.org/10.1109/TLT.2022.3180984>
- [9] S. S. Kumar, M. A. Lones, M. Maarek, and H. Zantout, “Navigating the landscape of automated feedback generation techniques for programming exercises,” *ACM Trans. Comput. Educ.*, vol. 25, no. 4, Oct. 2025. [Online]. Available: <https://doi.org/10.1145/3764593>
- [10] J. Ortega-Morla, A. Leis, A. Mallo, L. Morn-Fernndez, S. Guerreiro, A. Paz-Lpez, B. P rez-Snchez, N. Snchez-Maroto, A. Rodrguez-Arias, O. Fontenla-Romero, and F. Bellas, “Progtutor: A robotic-based framework to support teaching and learning of programming fundamentals,” *IEEE Transactions on Learning Technologies*, vol. 18, pp. 783–797, 2025. [Online]. Available: <https://doi.org/10.1109/TLT.2025.3598041>
- [11] R. Francisco and F. Silva, “Intelligent tutoring system for computer science education and the use of artificial intelligence: A literature review,” pp. 338–345, 2022. [Online]. Available: <http://dx.doi.org/10.5220/0011084400003182>
- [12] K. Chrysafiadi, M. Virvou, G. A. Tsihrintzis, and I. Hatzilygeroudis, “Evaluating the user’s experience, adaptivity and learning outcomes of a fuzzy-based intelligent tutoring system for computer programming for academic students in greece,” *Education and Information Technologies*, vol. 28, no. 6, pp. 6453–6483, Nov. 2022. [Online]. Available: <http://dx.doi.org/10.1007/s10639-022-11444-3>
- [13] M. Pankiewicz and R. S. Baker, “Large language models (gpt) for automating feedback on programming assignments,” 2023. [Online]. Available: <https://arxiv.org/abs/2307.00150>
- [14] S. Nayak, R. Agarwal, and S. K. Khatri, “Automated assessment tools for grading of programming assignments: A review,” pp. 1–4, 2022. [Online]. Available: <https://doi.org/10.1109/ICCCI54379.2022.9740769>
- [15] J.-Y. Kuo, H.-C. Lin, P.-F. Wang, and Z.-G. Nie, “A feedback system supporting students approaching a high-level programming course,” *Applied Sciences*, vol. 12, no. 14, 2022. [Online]. Available: <https://www.mdpi.com/2076-3417/12/14/7064>
- [16] I. Estvez-Ayres, P. Callejo, M. Hombrados-Herrera, C. Alario-Hoyos, and C. Delgado Kloos, “Evaluation of llm tools for feedback generation in a course on concurrent programming,” pp. 774–790, 2024. [Online]. Available: <https://doi.org/10.1007/s40593-024-00406-0>
- [17] T. Phung, V.-A. Pădurean, A. Singh, C. Brooks, J. Cambronero, S. Gulwani, A. Singla, and G. Soares, “Automating human tutor-style programming feedback: Leveraging gpt-4 tutor model for hint generation and gpt-3.5 student model for hint validation,” pp. 12–23, 2024. [Online]. Available: <https://doi.org/10.1145/3636555.3636846>
- [18] E. Frankford, C. Sauerwein, P. Bassner, S. Krusche, and R. Breu, “Ai-tutoring in software engineering education,” pp. 309–319, 2024. [Online]. Available: <https://doi.org/10.1145/3639474.3640061>
- [19] M. Messer, N. C. C. Brown, M. Kölling, and M. Shi, “Automated grading and feedback tools for programming education: A systematic review,” *ACM Trans. Comput. Educ.*, vol. 24, no. 1, Feb. 2024. [Online]. Available: <https://doi.org/10.1145/3636515>